Serving Ruby Code to the Web

Ruby is commonly used as the back end of a web application. In this pattern, a request is made to a web server, which executes Ruby code. The Ruby code returns HTML (usually), which is sent back to the web browser as the response.

While you could do all this in plain Ruby that you write yourself, there's no need for you to do all that work. The Ruby ecosystem has multiple web servers, including Puma,[3] Unicorn,[4] Thin,[5] and Falcon.[6] And there are also multiple web frameworks, including Ruby on Rails,[7] Sinatra,[8] Roda,[9] and Hanami.[10]

You might look at that incomplete list of web servers and frameworks and think that compatibility between servers and frameworks might be a nightmare of continually having to adjust the framework code based on what server you are using or vice versa. In fact, though, there is no nightmare here, and it's even possible for behavior to be written once and shared between multiple web servers. And the reason why this compatibility is not a problem is because of a library called Rack.[11]

Rack and Web Servers

Rack is a minimal interface for the relationship between a web server and a web application framework. Imagine you are writing a web server that wants to work with an application framework. There are certain logistical details of the relationship between the server and the framework that exist no matter the details of either tool's internal structure.

A user request comes as a set of text defined by the HTTP specification. The web server's job is to convert that text into Ruby objects and pass them to the framework. The framework's job is take that object, do something with it, and return Ruby objects to the web server, which converts them back to HTTP text or data to send to the browser.

Rack provides:

- A standard structure for the environment data coming into the web server via the user request. Specifically, a hash with a pre-defined set of keys.
- A standard structure for the response data coming back to the web server from the frameworks. The response is an array with three elements: the return status, the headers, and the body.
- A mechanism for the interface between the two. A rack app is a method or block that takes the environment as input and returns the response as output. Rack provides the method `run`, which takes rack apps, sends them environments, and returns the result.

Rack also provides a structure for chaining Rack applications together, so you might have a series of small rack applications, one of which might do, say, authentication, another of which might sanitize input, and another of which might clean up image sizing. Rack makes

3. https://puma.io
4. https://yhbt.net/unicorn
5. https://github.com/macournoyer/thin
6. https://github.com/socketry/falcon
7. https://rubyonrails.org
8. https://sinatrarb.com
9. https://github.com/jeremyevans/roda
10. https://hanamirb.org
11. https://github.com/rack/rack